CHAP-4_BAYESIAN

Tawhid Tasdique

2025-12-24

Ex-4.1

The posterior distribution corresponding to the 2nd sample survey is $Beta(31, 21)$ which was computed from the general posterior distribution $Beta(a + (\sum_{i=1}^{n} y_i),\ b + n - (\sum_{i=1}^{n} y_i))$ corresponding to the general prior distribution $Beta(a, b)$. The posterior distribution corresponding to the 1st sample survey from $Ex - 3.1$ is $Beta(58, 44)$. Using posterior distributions we will estimate $Pr(\theta_1 < \theta_2 \mid the\ data\ and\ prior)$. The solution is simpler than I thought it would be. I initially assumed I would have to derive the pdf of difference between the two posterior random variables corresponding to θ_1 and θ_2.

```r
set.seed(27)

theta1_mc <- rbeta(n = 5000, shape1 = 58, shape2 = 44)

theta2_mc <- rbeta(n = 5000, shape1 = 31, shape2 = 21)

# answer
mean(theta1_mc < theta2_mc)
```

```
## [1] 0.6194
```

Ex-4.2

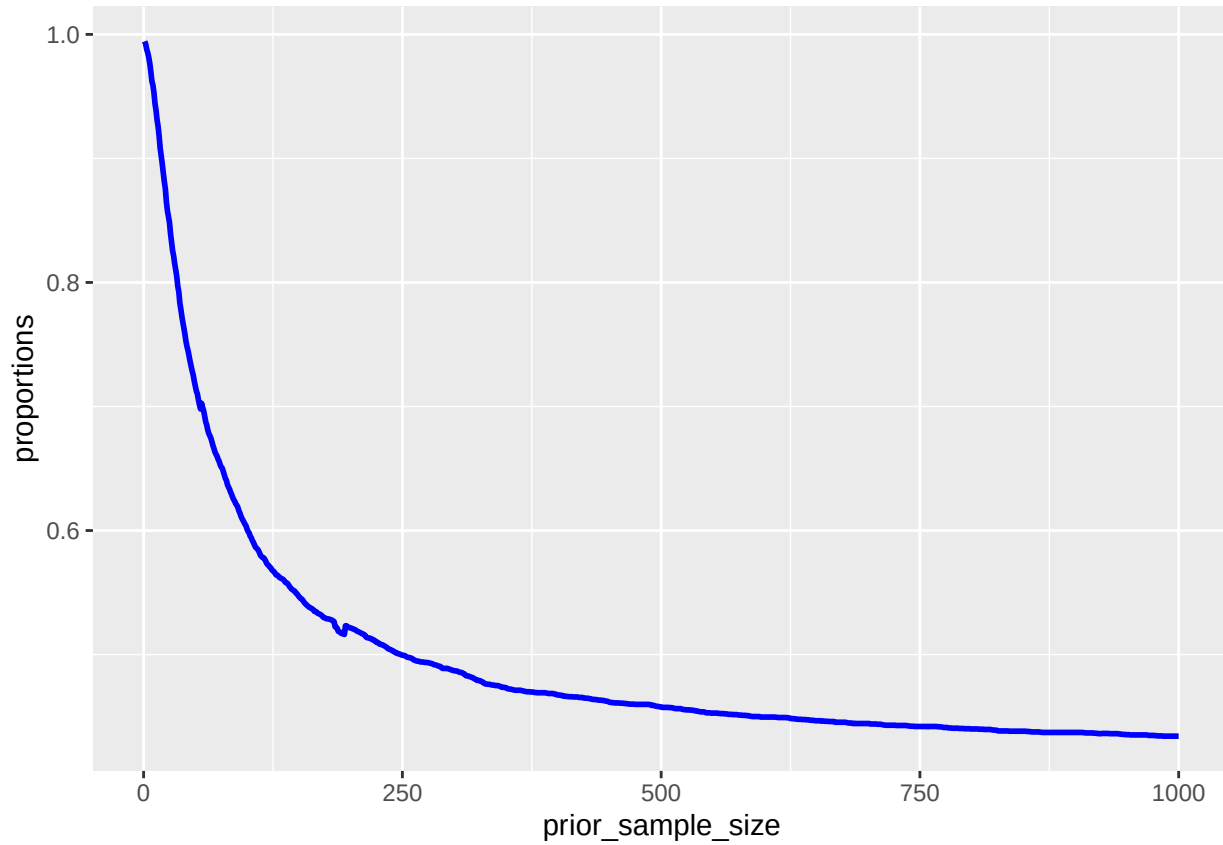
Ex-4.2(a)

```r
set.seed(27)

# Sampling from Posterior A
mcA <- rgamma(n = 5000, shape = 237, rate = 20)

# Sampling from Posterior B
mcB <- rgamma(n = 5000, shape = 125, rate = 14)

# required probability estimate
mean(mcB < mcA)
```

```
## [1] 0.9944
```

Ex-4.2(b)

```r
# prior sample size for strain B
n0 <- 1:1000

# function for computing proportion of successful events
props <- sapply(
  X = n0,
  FUN = function(n) {
    set.seed(27)

    # Sampling from Posterior A
    mcA <- rgamma(n = 5000, shape = 237, rate = 20)

    # Sampling from Posterior B
    mcB <- rgamma(n = 5000,
                  shape = (12 * n + 113),
                  rate = (n + 13))

    # required probability estimate
    mean(mcB < mcA)

  }
)

# graphics
tibble(prior_sample_size = n0, proportions = props) %>%
  ggplot(aes(x = prior_sample_size, y = proportions)) +
  geom_line(color = "blue",
            linewidth = 1)
```



Ex-4.2(c)

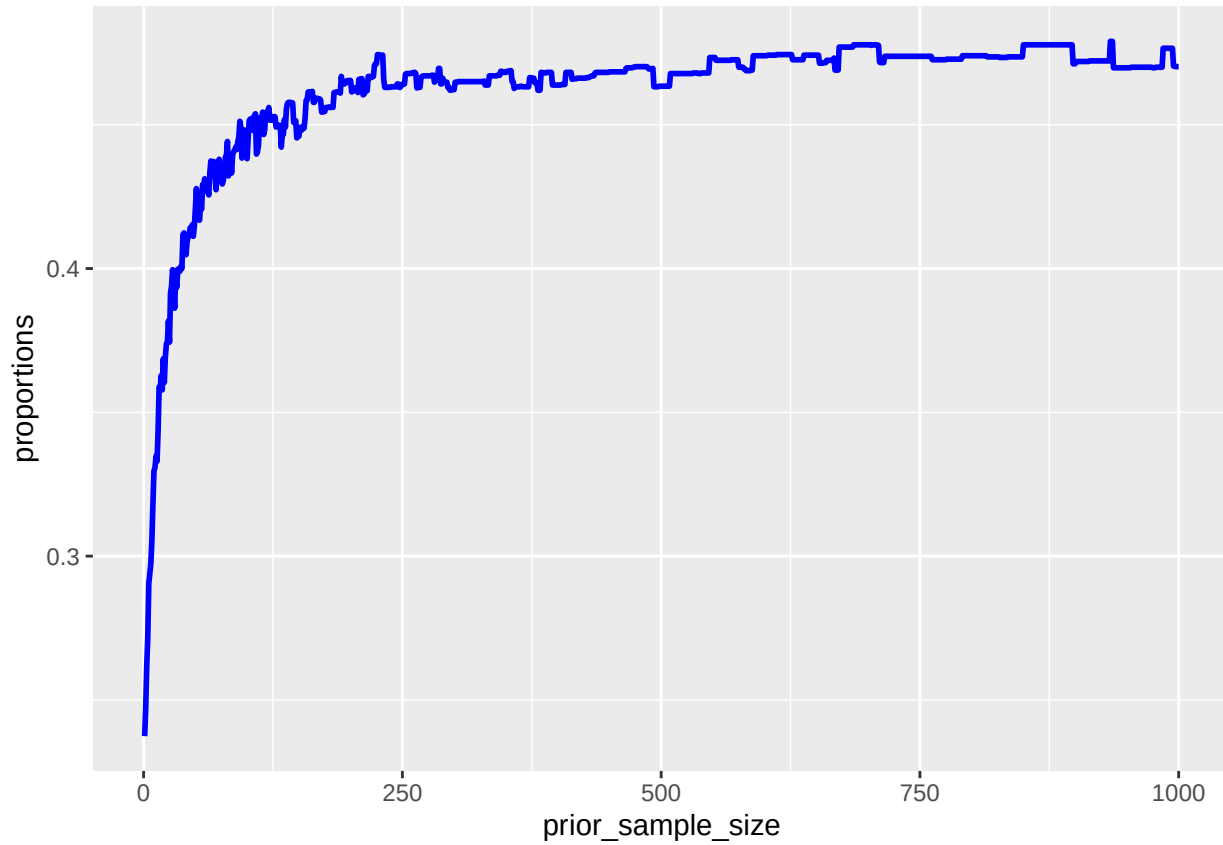
The posterior distributions were computed by hand and the procedures can be found in the **HomeCopy-I**. Information acquired from $Ex - 3.3$ is mentioned on sticky notes instead of repeating derivations.

```
################################################################

# Repeating for part(a)

set.seed(27)

# predicting 5000 values for strain A
A_pred_a <- rnbinom(n = 5000, size = 237, prob = (20/21))

# predicting 5000 values for strain B from part(a)'s distribution
B_pred_a <- rnbinom(n = 5000, size = 125, prob = (14/15))

# Answer for part(a): required probability estimate
mean(B_pred_a < A_pred_a)
```

```
## [1] 0.6918
```

```
################################################################
```

```r
# Repeating for part(b)

# generating prior sample sizes for part(b)
n_0 <- 1:1000

# function for computing proportions of successful events corresponding to part(b)
props <- sapply(
  X = n_0,
  FUN = function(n0) {
    set.seed(27)

    # Sampling from Posterior Predictive of Strain A
    A_pred_b <- rnbinom(n = 5000, size = 237, prob = (20/21))

    # Sampling from Posterior Predictive of strain B
    B_pred_b <- rnbinom(n = 5000,
                        size = (12*n0 + 113),
                        prob = ((n0 + 13)/(n0 + 14)))

    # required probability estimate
    mean(A_pred_b < B_pred_b)

  }
)

# Answer for part(b)
# graphics
tibble(prior_sample_size = n_0, proportions = props) %>%
  ggplot(aes(x = prior_sample_size, y = proportions)) +
  geom_line(color = "blue",
            linewidth = 1)
```



Ex-4.3

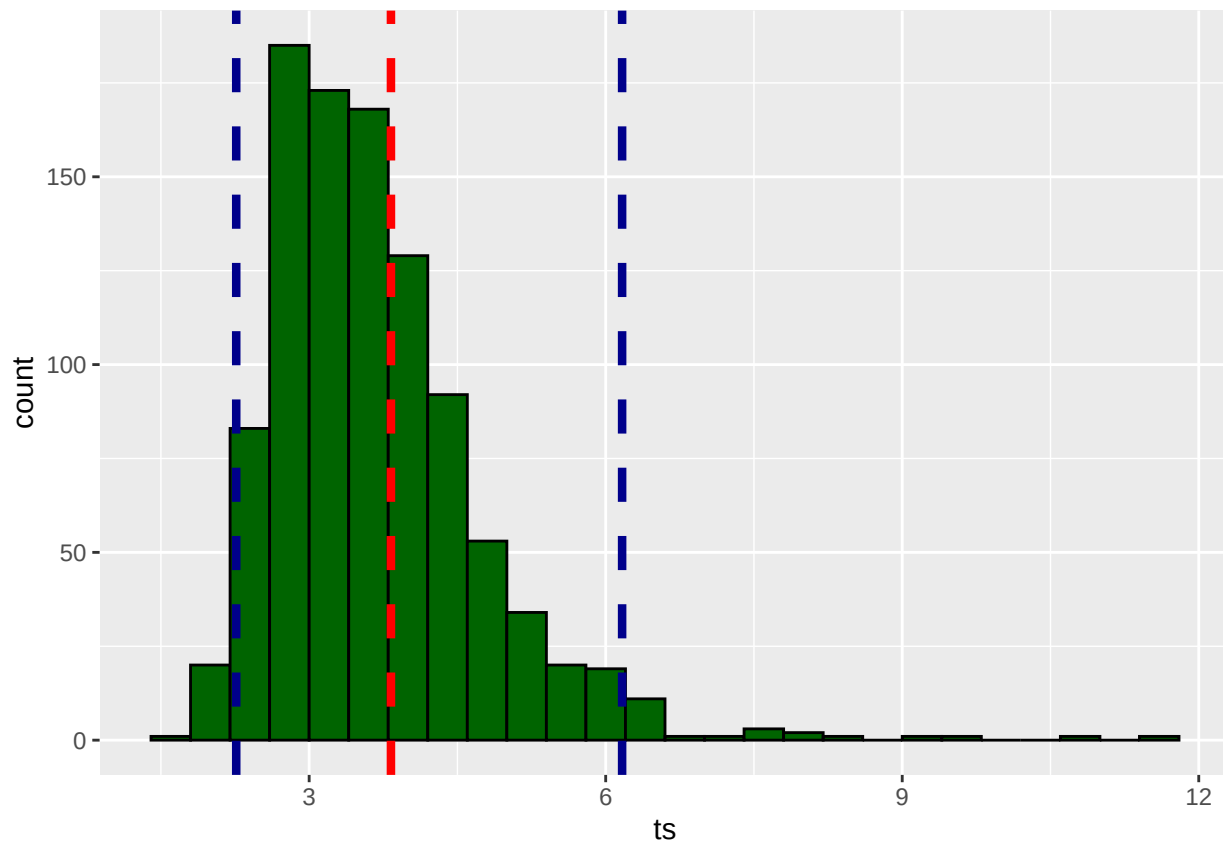
Ex-4.3(a)

They want me to **run posterior predictive checks**, **to investigate the adequacy of the given (Poisson) sampling model**, **to assess the fit of the Poisson model for these data**.

These are all just fancy ways of telling me to check whether the sample or data supports our assumed Poisson sampling model. We're operating under the assumption that our data came from a Poisson sampling model. If that is truly the case then our observed sample statistic would not be exceptional in comparison to the rest of the statistics simulated by our posterior predictive distribution.

```
# strain A data
yA <- c(12, 9, 12, 14, 13, 13, 15, 8, 15, 6)

# observed sample statistic
T_stat <- mean(yA)/sd(yA)

# prescribed sample size
nA <- 10

set.seed(27)

# simulated statistics
```

```r
ts <- sapply(
  X = 1:1000,
  FUN = function(b) {

    a <- rnbinom(n = nA,
                 size = 237,
                 prob = (20 / 21))

    mean(a)/sd(a)
  }
)

# graphics
tibble(stat = ts) %>%
  ggplot(aes(x = ts)) +
  geom_histogram(binwidth = 0.4,
                 # (max - min)/20 recommended width for continuous data
                 fill = "darkgreen",
                 color = "black") +
  geom_vline(xintercept = T_stat, # observed sample statistic
             color = "red",
             linetype = "dashed",
             linewidth = 1.5) +
  geom_vline(xintercept = quantile(x = ts,
                                   probs = 0.025),
             color = "darkblue",
             linetype = "dashed",
             linewidth = 1.5) +
  geom_vline(xintercept = quantile(x = ts,
                                   probs = 0.975),
             color = "darkblue",
             linetype = "dashed",
             linewidth = 1.5)
```



Conclusion

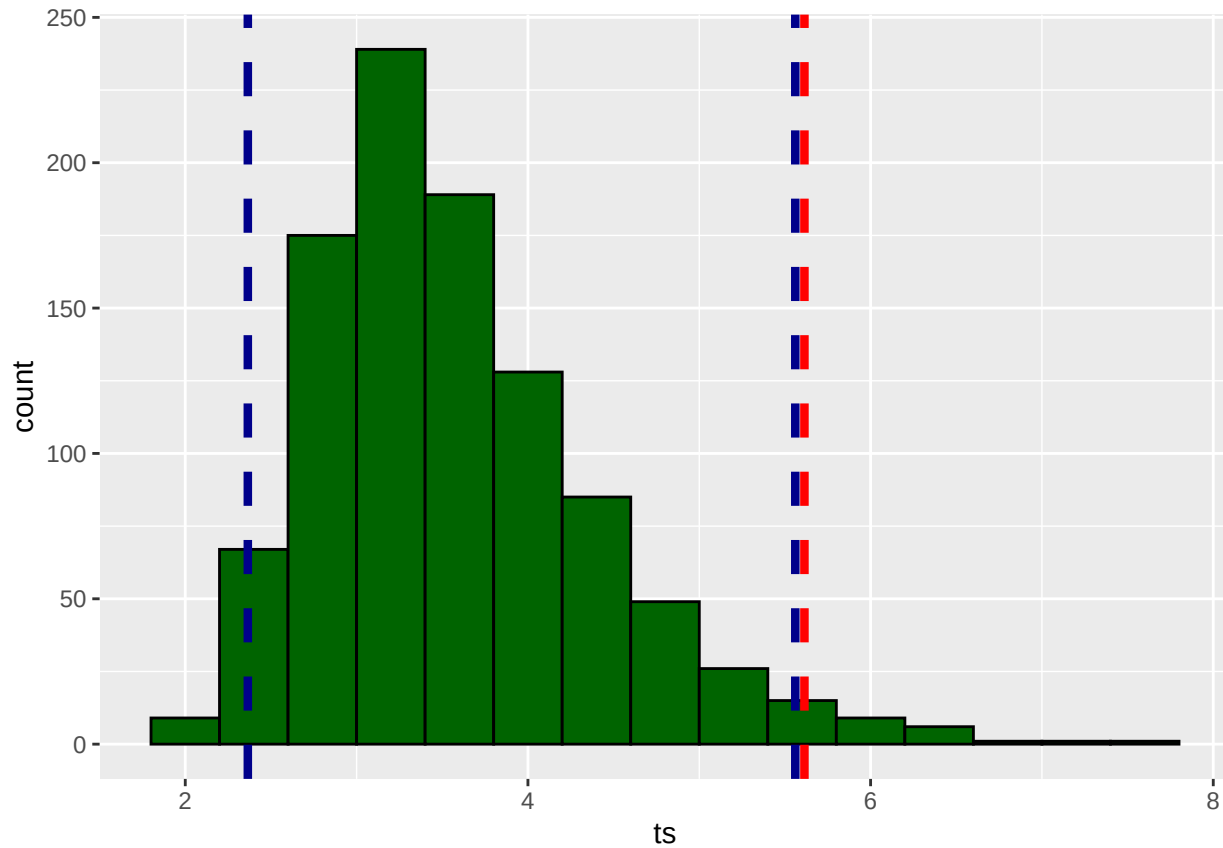
It appears that the observed sample statistic falls within the quantile-based 95% confidence interval of the posterior predictive distribution of our statistic of interest. Hence, it is not uncommon relative to the main bulk of the produced sampling distribution. And so, we conclude that the Poisson sampling model is **adequate**.

Ex-4.3(b)

We're running the same posterior predictive check for data from strain B.

```
# strain B data
yB <- c(11, 11, 10, 9, 9, 8, 7, 10, 6, 8, 8, 9, 7)

# observed sample statistic
T_stat <- mean(yB)/sd(yB)

# prescribed sample size
nB <- 13

set.seed(27)

# simulated statistics
ts <- sapply(
```

```r
  X = 1:1000,
  FUN = function(b) {

    a <- rnbinom(n = nB,
                 size = 237,
                 prob = (20 / 21))

    mean(a)/sd(a)
  }
)

# graphics
tibble(stat = ts) %>%
  ggplot(aes(x = ts)) +
  geom_histogram(binwidth = 0.4,
                 # (max - min)/20 recommended width for continuous data
                 fill = "darkgreen",
                 color = "black") +
  geom_vline(xintercept = T_stat, # observed sample statistic
             color = "red",
             linetype = "dashed",
             linewidth = 1.5) +
  geom_vline(xintercept = quantile(x = ts,
                                   probs = 0.025),
             color = "darkblue",
             linetype = "dashed",
             linewidth = 1.5) +
  geom_vline(xintercept = quantile(x = ts,
                                   probs = 0.975),
             color = "darkblue",
             linetype = "dashed",
             linewidth = 1.5)
```



Conclusion

It appears that the observed sample statistic falls outside the quantile-based 95% confidence interval of the posterior predictive distribution of our statistic of interest. Hence, it is uncommon relative to the main bulk of the produced sampling distribution. And so, we conclude that the Poisson sampling model is **not adequate**.

Ex-4.4

Ex-4.4(a)

Discrete approximation is taught in ch-6 which is admittedly, a little advanced compared to ch-4. We will use the **discretized** posterior distribution for **interval estimation**. We resort to this method because the actual posterior pdf has a **non-standard** form or an **unrecognized** form, making it difficult to compute the interval.

Discrete Approximation for Interval Estimation

We're gonna compute the 95% quantile-based confidence interval.

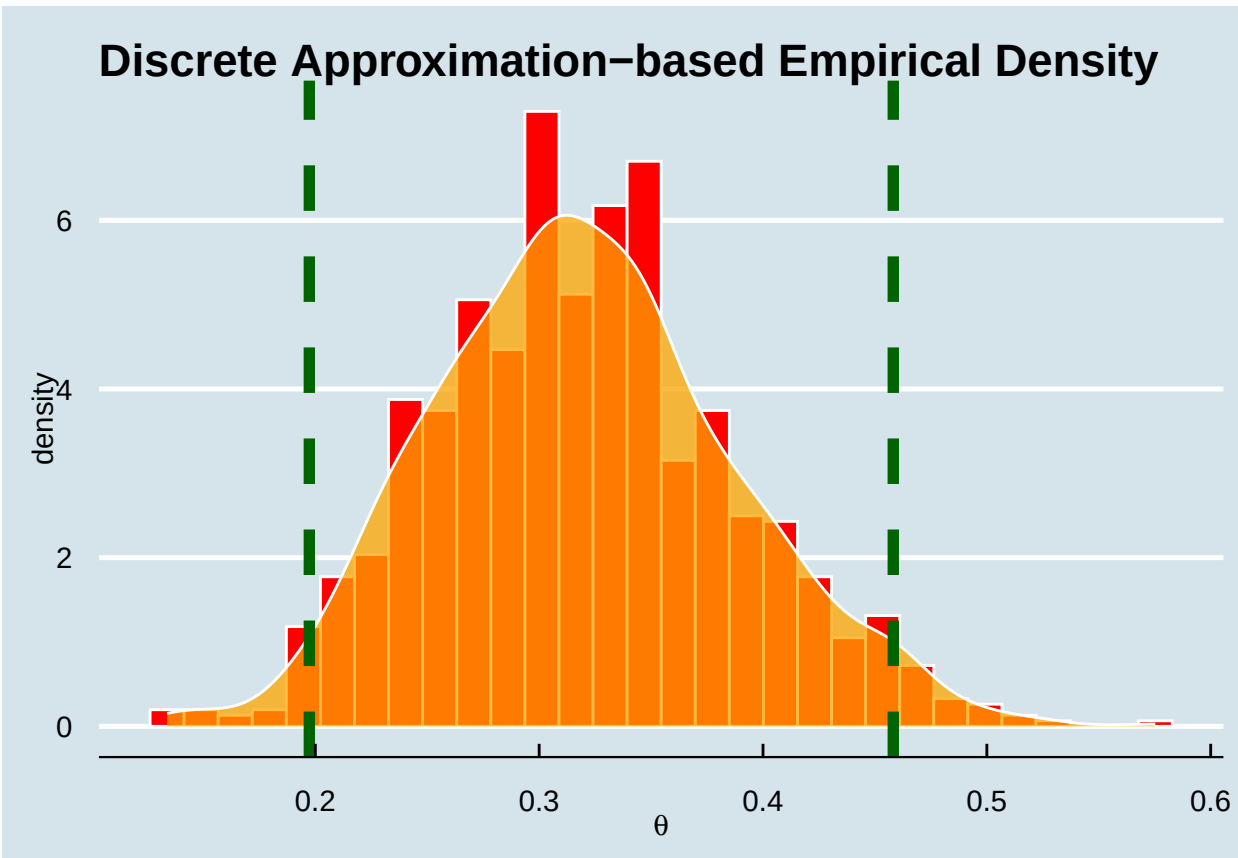
```
# discrete set of parameter values, very dense sequence
D <- seq(from = 0, to = 1, length.out = 300)
```

```r
# given
n <- 43
y <- 15

#################################################################

# discrete approximation of mixed posterior

# non-marginalized mixed posterior
mixed.post <- 3*(D^(y+1))*((1-D)^(n+7-y)) + (D^(y+7))*((1-D)^(n+1-y))

# standardized mixed posterior
mixed.post <- mixed.post/sum(mixed.post)

# setting seed
set.seed(27)

# sampling with replacement from the discretized parameter space
sample_disc <- sample(x = D,
                      size = 1000,
                      replace = T,
                      prob = mixed.post)
# what sampling procedure is this??
# It's exactly like sampling from a multinomial distribution which is in fact a discrete distribution.

#################################################################

# 95% quantile-based interval
(
disc.approx.CI <- c(
  Lower = quantile(x = sample_disc, probs = 0.025),
  Upper = quantile(x = sample_disc, probs = 0.975)
)
)
```

```
##  Lower.2.5% Upper.97.5%
##   0.1973244   0.4581940
```

```r
#################################################################

# graphics
tibble(
  values = sample_disc
) %>%
  ggplot(aes(x = values)) +
  geom_histogram(
    aes(y = after_stat(density)), # relative frequency histogram
    bins = 30,
    fill = "red",
    color = "white"
  ) +
  geom_density(
    fill = "orange",
```

```
    color = "white",
    alpha = 0.75
  ) +
  geom_vline(
    xintercept = disc.approx.CI,
    color = "darkgreen",
    linetype = "dashed",
    linewidth = 2
  ) +
  xlab(label = expression(theta)) +
  labs(title = "Discrete Approximation-based Empirical Density") +
  theme_economist()
```



Plotting the Actual Mixed Posterior

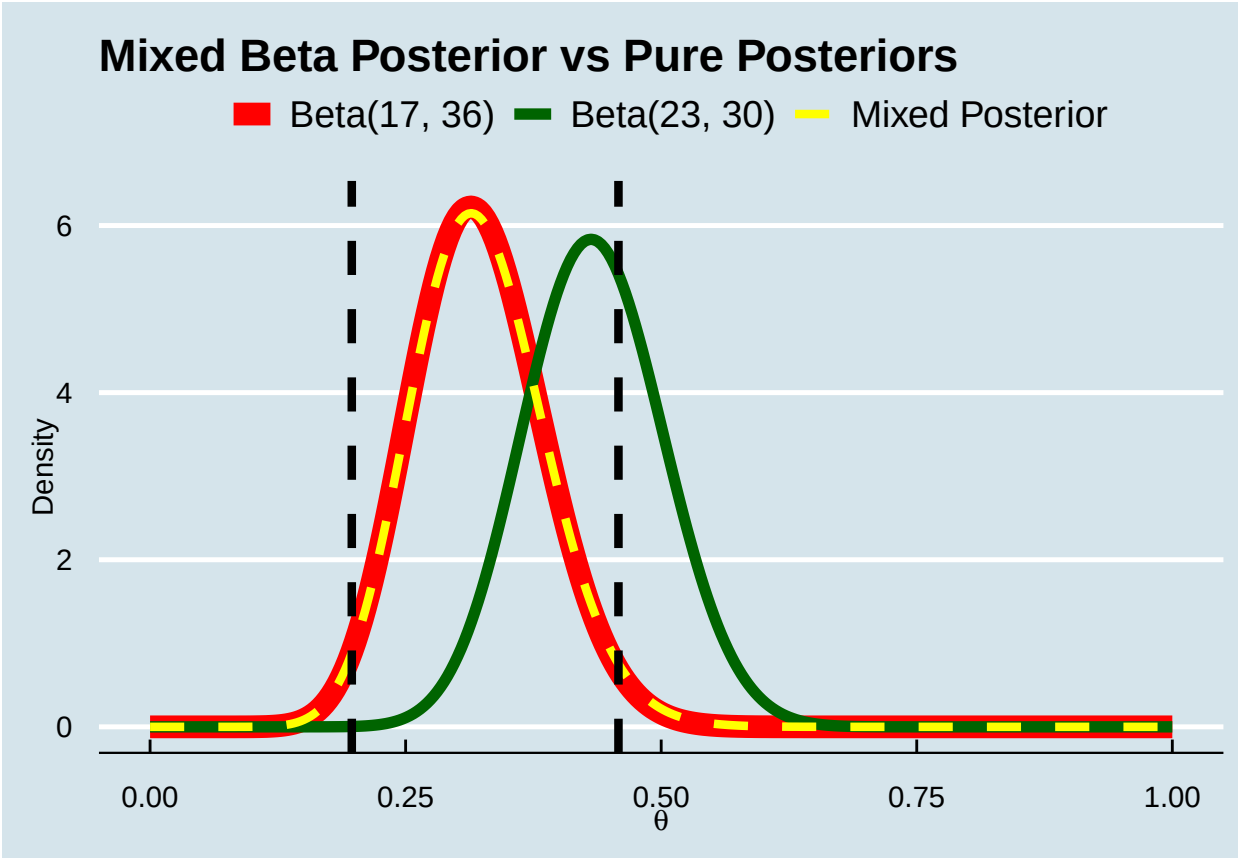
We have the actual pdf at hand, derived in the **HomeCopy-II**. Hence, there is no need for discrete approximation with regards to plotting the density.

```
# discrete set of parameter values, very dense sequence
D <- seq(from = 0, to = 1, length.out = 300)

# pdf of mixed posterior
dMixedPosterior <- function(theta, n = 43, y = 15){
```

```r
  # constant
  `C*` <- (
    3*beta(a = (y+2), b = (n+8-y)) + beta(a = (y+8), b = (n+2-y))
  )

  # weight for 1st component
  w1 <- (3 / `C*`) * beta(a = (y+2), b = (n+8-y))

  # weight for 2nd component
  w2 <- (1 / `C*`) * beta(a = (y+8), b = (n+2-y))

  # 1st component of mixed posterior pdf
  post1 <- dbeta(x = theta,
                 shape1 = (y + 2),
                 shape2 = (n + 8 - y))

  # 2nd component of mixed posterior pdf
  post2 <- dbeta(x = theta,
                 shape1 = (y + 8),
                 shape2 = (n + 2 - y))

  # mixed posterior pdf (weighted sum)
  posteriorPDF <- w1*post1 + w2*post2

  return(data.frame(post1, post2, posteriorPDF))

}
```

```r
# plotting begins
tibble(
  theta = D,
  post1 = dMixedPosterior(theta = theta)$post1,
  post2 = dMixedPosterior(theta = theta)$post2,
  mixedPDF = dMixedPosterior(theta = theta)$posteriorPDF
) %>%
  ggplot(aes(x = theta)) +

  # 1st pure posterior density
  geom_line(
    aes(
      y = post1,
      color = "Beta(17, 36)",
      linetype = "Beta(17, 36)",
      linewidth = "Beta(17, 36)"
    ),
  ) +

  # 2nd pure posterior density
  geom_line(
    aes(
      y = post2,
      color = "Beta(23, 30)",
      linetype = "Beta(23, 30)",
```

```r
    linewidth = "Beta(23, 30)"
  ),
) +

# mixed posterior density
geom_line(
  aes(
    y = mixedPDF,
    color = "Mixed Posterior",
    linetype = "Mixed Posterior",
    linewidth = "Mixed Posterior"
  ),
) +

# 95% confidence interval via discrete approximation
geom_vline(
  xintercept = disc.approx.CI,
  color = "black",
  linetype = "dashed",
  linewidth = 1.5
) +

scale_color_manual(values = c(
  "Beta(17, 36)" = "red",
  "Beta(23, 30)" = "darkgreen",
  "Mixed Posterior" = "yellow"
)) +
scale_linetype_manual(
  values = c(
    "Beta(17, 36)" = "solid",
    "Beta(23, 30)" = "solid",
    "Mixed Posterior" = "dashed"
  )
) +
scale_linewidth_manual(values = c(
  "Beta(17, 36)" = 4,
  "Beta(23, 30)" = 2,
  "Mixed Posterior" = 1.5
)) +
labs(color = "",
     linetype = "",
     linewidth = "",
     title = "Mixed Beta Posterior vs Pure Posteriors"
     ) +
ylab(label = "Density") +
xlab(label = expression(theta)) +
theme_economist()
```



Conclusion

Because the 1st pure posterior, $Beta(17,\ 36)$, has more weight relative to the 2nd pure posterior, $Beta(23,\ 30)$, it appears that the mixed posterior almost superimposes itself upon the 1st pure posterior i.e. it is biased towards the 1st pure posterior.

The question asked to plot just the mixed posterior but I'm also leaving code for the pure posteriors (to be displayed on the same canvas) for the benefit of comparison (and fun).

Also, the discrete approximation-based interval appears to appreciably contain the bulk of theoretical mixed posterior distribution.

Ex-4.4(b)

We will construct a Monte-Carlo approximation of the mixed posterior distribution plotted in $EX-4.4(A)$. For that we need to define a random number generators like `rbinom` or `rgamma`. Once the numbers have been drawn, we'll plot them via an empirical histogram or an empirical density graph.

```
# random number generating function
rMixedPosterior <- function(size = 1000, n = 43, y = 15){

  # for storage purpose
  number <- NULL

  # constant
```

```r
  `C*` <- (
    3*beta(a = (y+2), b = (n+8-y)) + beta(a = (y+8), b = (n+2-y))
  )

  # the probability from the 1st pure posterior
  w <- (3 / `C*`) * beta(a = (y+2), b = (n+8-y))

  for(i in 1:size){

    if(rbinom(n = 1, size = 1, prob = w) == 1){

      number[i] <- rbeta(n = 1, shape1 = (y+2), shape2 = (n+8-y))

    }
    else{

      number[i] <- rbeta(n = 1, shape1 = (y+8), shape2 = (n+2-y))

    }

  }

  return(number)

}
```

```r
set.seed(27)

# random numbers generated for 4(b)
rn4b <- rMixedPosterior(size = 10000)

# Monte-Carlo approximated quantile-based 95% confidence interval
(
mc.approx.CI <- c(
  Lower = quantile(x = rn4b, probs = 0.025),
  Upper = quantile(x = rn4b, probs = 0.975)
)
)
```

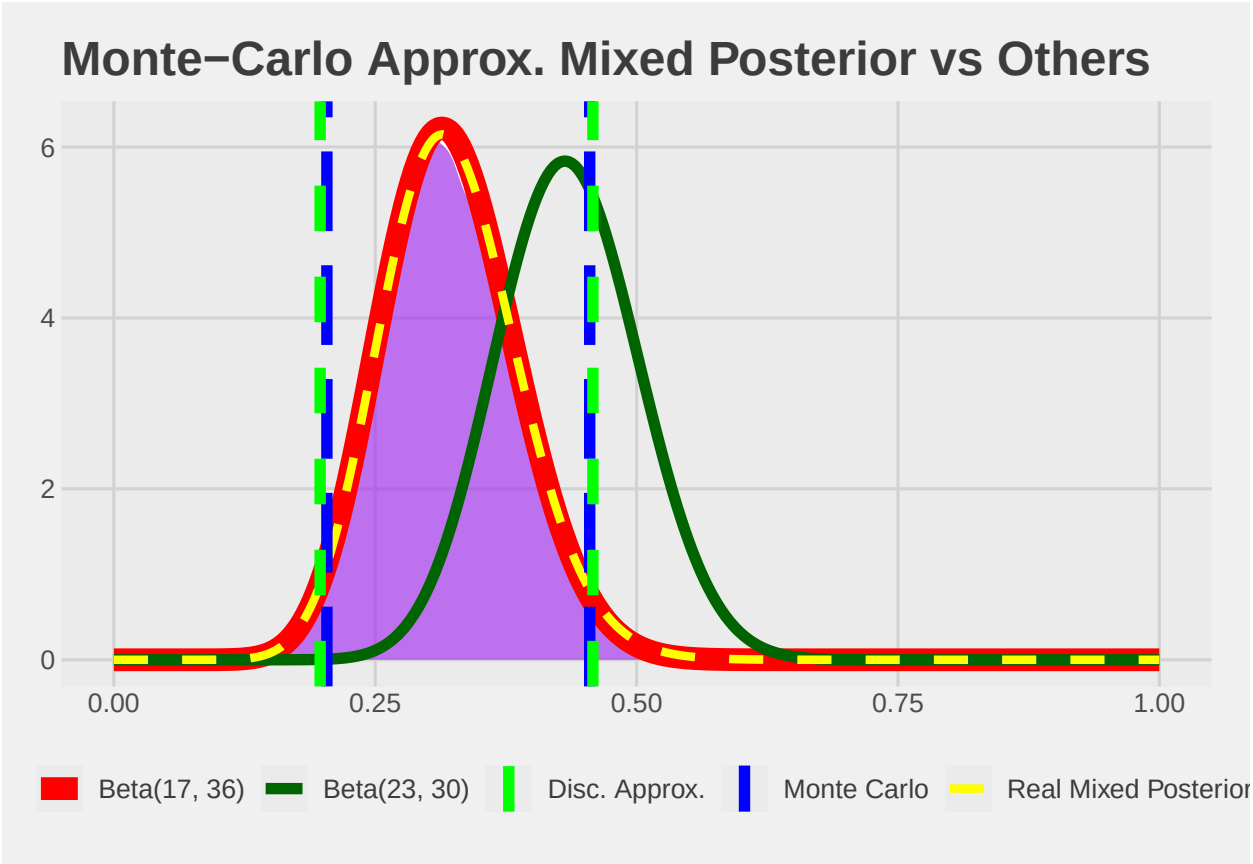
```
##  Lower.2.5% Upper.97.5%
##   0.2037706   0.4551339
```

```r
posteriors_4a <- tibble(
  theta = D,
  post1 = dMixedPosterior(theta = theta)$post1,
  post2 = dMixedPosterior(theta = theta)$post2,
  mixedPDF = dMixedPosterior(theta = theta)$posteriorPDF
)
```

```r
# empirical density graph
tibble(zahlen = rn4b) %>%
  ggplot(aes(x = zahlen)) +
```

```r
# Monte-Carlo generated posterior
geom_density(
  fill = "purple",
  color = "white",
  alpha = 0.6
) +

# Beta(17, 36)
geom_line(
  data = posteriors_4a,
  mapping = aes(
    x = theta,
    y = post1,
    color = "Beta(17, 36)",
    linetype = "Beta(17, 36)",
    linewidth = "Beta(17, 36)"
  )
) +

# Beta(23, 30)
geom_line(
  data = posteriors_4a,
  mapping = aes(
    x = theta,
    y = post2,
    color = "Beta(23, 30)",
    linetype = "Beta(23, 30)",
    linewidth = "Beta(23, 30)"
  )
) +

# Mixed Beta posterior
geom_line(
  data = posteriors_4a,
  mapping = aes(
    x = theta,
    y = mixedPDF,
    color = "Real Mixed Posterior",
    linetype = "Real Mixed Posterior",
    linewidth = "Real Mixed Posterior"
  )
) +

# Monte Carlo CI (2 lines)
geom_vline(
  data = tibble(mc.CI = mc.approx.CI),
  aes(color = "Monte Carlo",
      linetype = "Monte Carlo",
      linewidth = "Monte Carlo",
      xintercept = mc.CI)
) +

# Discrete Approximation CI (2 lines)
```

```r
geom_vline(
  data = tibble(disc.CI = disc.approx.CI),
  aes(color = "Disc. Approx.",
      linetype = "Disc. Approx.",
      linewidth = "Disc. Approx.",
      xintercept = disc.CI)
) +

scale_color_manual(values = c(
  "Disc. Approx." = "green",
  "Monte Carlo" = "blue",
  "Beta(17, 36)" = "red",
  "Beta(23, 30)" = "darkgreen",
  "Real Mixed Posterior" = "yellow"
)) +
scale_linetype_manual(values = c(
  "Disc. Approx." = "dashed",
  "Monte Carlo" = "longdash",
  "Beta(17, 36)" = "solid",
  "Beta(23, 30)" = "solid",
  "Real Mixed Posterior" = "dashed"
)) +
scale_linewidth_manual(values = c(
  "Disc. Approx." = 2,
  "Monte Carlo" = 2,
  "Beta(17, 36)" = 4,
  "Beta(23, 30)" = 2,
  "Real Mixed Posterior" = 1.5
)) +
labs(title = "Monte-Carlo Approx. Mixed Posterior vs Others",
     color = "",
     linetype = "",
     linewidth = "") +
xlab(expression(theta)) +
ylab("Density") +
ggthemes::theme_fivethirtyeight()
```



Conclusion

Clearly, the confidence intervals from **Monte-Carlo approximation** and **discrete approximation** appear almost identical. The same can be said about the **Theoretical mixed posterior** (in yellow) and the **Monte-Carlo approximated posterior** (in purple). Therefore, the results in part-(a) and part-(b) coincide remarkably.

Ex-4.5

Ex-4.5(b)

```
cancer_noreact <- read.csv("cancer_noreact.dat", sep="")

cancer_react <- read.csv("cancer_react.dat", sep="")
```

The general form of the posterior distribution was derived in $Ex-4.5(a)$. Now, we just compute the sums specific to the data and pour them into the mold. We still won't have specific distributions because the hyper-parameters $a_1,\ b_1,\ a_2,\ b_2$ are unknown to us.

```
cancer_noreact %>%
  reframe(
    `Sum of Y` = sum(y),
```

```r
    `Sum of X` = sum(x)
  )
```

```
##   Sum of Y Sum of X
## 1     2285     1037
```

```r
cancer_react %>%
  reframe(
    `Sum of Y` = sum(y),
    `Sum of X` = sum(x)
  )
```

```
##   Sum of Y Sum of X
## 1      256       95
```

Conclusion

We know from 4.5(a), $(\theta|X,Y) \sim Beta(a + (\sum_{i=1}^{n} Y_i),\ b + (\sum_{i=1}^{n} X_i))$. Hence, the required posteriors are as follows:

- $\theta_1|X,Y \sim Beta(a_1 + 2285,\ b_1 + 1037)$
- $\theta_2|X,Y \sim Beta(a_2 + 256,\ b_2 + 95)$

Ex-4.5(c)

Posterior means and quantile-based confidence intervals are easy to compute since posterior distributions are of recognizable/standard form.

Hence, one might believe that there is no need for approximation on those fronts. However, the posterior distribution, which partly depends upon the data, is consequentially an empirical quantity. Therefore, unless specifically mentioned otherwise, the posterior quantile-based intervals require computing empirical quantiles, not theoretical quantiles.

Estimating $Pr(\theta_2 > \theta_1|\ data)$ is definitely going to require Monte-Carlo approximation.

Ex-4.5(c)(i)

```r
# given
a1 <- a2 <- 2.2*100
b1 <- b2 <- 100

# posterior means
(
post.mean.1 <- (a1 + 2285)/(b1 + 1037)
)
```

```
## [1] 2.203166
```

```r
(
post.mean.2 <- (a2 + 256)/(b1 + 95)
)
```

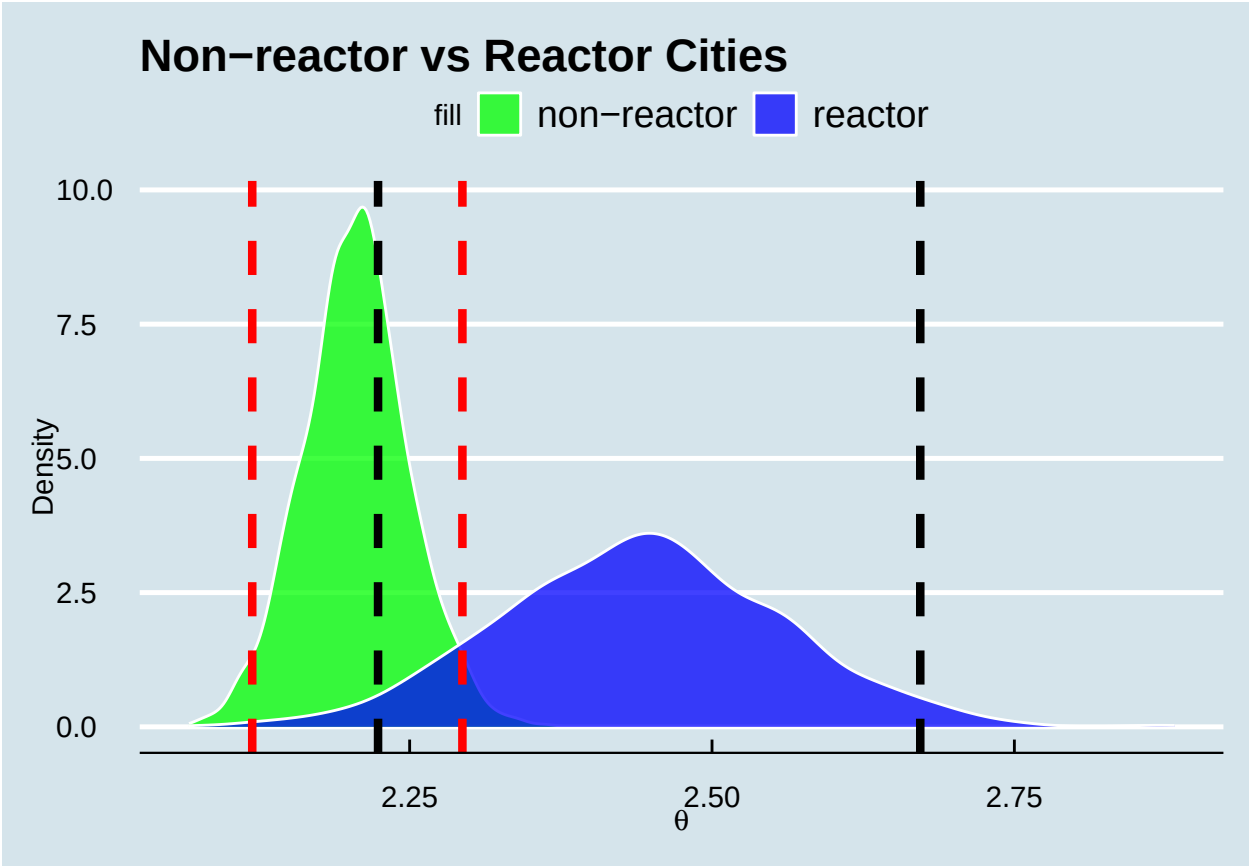
```
## [1] 2.441026
```

```r
# setting seed
set.seed(27)

# required posterior probability estimate
(
`Pr(theta2 > theta1 | data)` <- mean(
  x = (
    rgamma(n = 1000,
           shape = (a2+256),
           rate = (b2+95)) > rgamma(n = 1000,
                                    shape = (a1+2285),
                                    rate = (b1+1037))
  )
)
)
```

```
## [1] 0.977
```

```r
##########################################################

# setting seed
set.seed(27)

# data
dta <- tibble(
  # non-reactor cities
  nonreactor = rgamma(
    n = 1000,
    shape = (a1+2285),
    rate = (b1+1037)
  ),

  # reactor cities
  reactor = rgamma(
    n = 1000,
    shape = (a2+256),
    rate = (b2+95)
  )
)

# graphics
dta %>%
  ggplot() +

  # M.C. approximation of 1st posterior
  geom_density(aes(x = nonreactor,
```

```r
                  fill = "non-reactor"),
              color = "white",
              alpha = 0.75) +

# M.C. approximation of 2nd posterior
geom_density(aes(x = reactor,
                  fill = "reactor"),
              color = "white",
              alpha = 0.75) +

# confidence interval for non-reactor cities
geom_vline(
  xintercept = dta$nonreactor %>%
    quantile(probs = c(0.025, 0.975)),
  color = "red",
  linetype = "dashed",
  linewidth = 1.5
) +

# confidence interval for reactor cities
geom_vline(
  xintercept = dta$reactor %>%
    quantile(probs = c(0.025, 0.975)),
  color = "black",
  linetype = "dashed",
  linewidth = 1.5
) +

scale_fill_manual(
  values = c(
    "non-reactor" = "green",
    "reactor" = "blue"
  )
) +
labs(title = "Non-reactor vs Reactor Cities") +
xlab(label = expression(theta)) +
ylab(label = "Density") +
theme_economist()
```



Ex-4.5(c)(ii)

```
# given
a1 <- 2.2*100; a2 <- 2.2
b1 <- 100; b2 <- 1

# posterior means
(
post.mean.1 <- (a1 + 2285)/(b1 + 1037)
)
```

```
## [1] 2.203166
```

```
(
post.mean.2 <- (a2 + 256)/(b1 + 95)
)
```

```
## [1] 1.324103
```

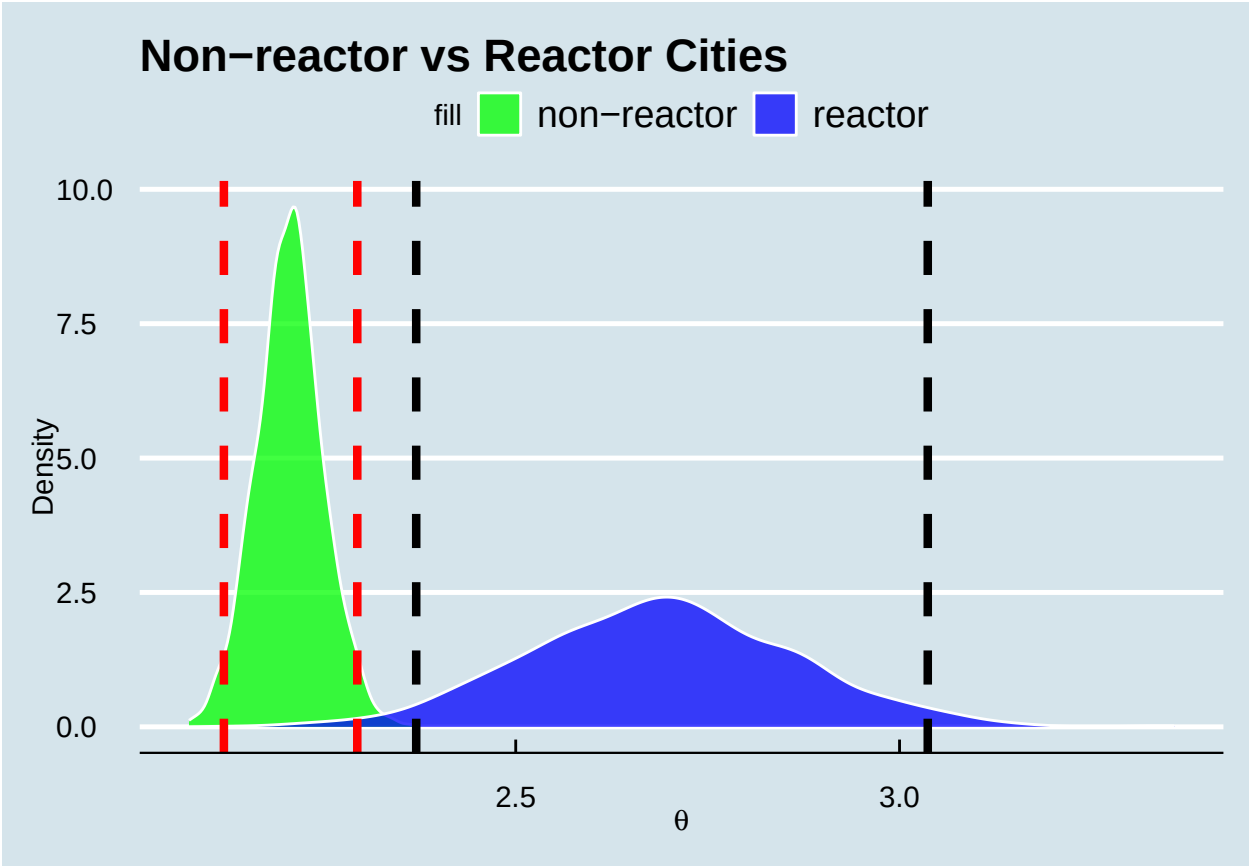
```
# setting seed
set.seed(27)

# required posterior probability estimate
```

```r
(
`Pr(theta2 > theta1 | data)` <- mean(
  x = (
    rgamma(n = 1000,
           shape = (a2+256),
           rate = (b2+95)) > rgamma(n = 1000,
                                    shape = (a1+2285),
                                    rate = (b2+1037))
  )
)
)
```

```
## [1] 0.958
```

```r
#######################################################

# setting seed
set.seed(27)

# data
dta <- tibble(
  # non-reactor cities
  nonreactor = rgamma(
    n = 1000,
    shape = (a1+2285),
    rate = (b1+1037)
  ),

  # reactor cities
  reactor = rgamma(
    n = 1000,
    shape = (a2+256),
    rate = (b2+95)
  )
)

# graphics
dta %>%
  ggplot() +

  # M.C. approximation of 1st posterior
  geom_density(aes(x = nonreactor,
                   fill = "non-reactor"),
               color = "white",
               alpha = 0.75) +

  # M.C. approximation of 2nd posterior
  geom_density(aes(x = reactor,
                   fill = "reactor"),
               color = "white",
               alpha = 0.75) +

  # confidence interval for non-reactor cities
```

```r
geom_vline(
  xintercept = dta$nonreactor %>%
    quantile(probs = c(0.025, 0.975)),
  color = "red",
  linetype = "dashed",
  linewidth = 1.5
) +

# confidence interval for reactor cities
geom_vline(
  xintercept = dta$reactor %>%
    quantile(probs = c(0.025, 0.975)),
  color = "black",
  linetype = "dashed",
  linewidth = 1.5
) +

scale_fill_manual(
  values = c(
    "non-reactor" = "green",
    "reactor" = "blue"
  )
) +
labs(title = "Non-reactor vs Reactor Cities") +
xlab(label = expression(theta)) +
ylab(label = "Density") +
theme_economist()
```



Ex-4.5(c)(iii)

```
# given
a1 <- a2 <- 2.2
b1 <- b2 <- 1

# posterior means
(
post.mean.1 <- (a1 + 2285)/(b1 + 1037)
)
```

```
## [1] 2.203468
```

```
(
post.mean.2 <- (a2 + 256)/(b1 + 95)
)
```

```
## [1] 2.689583
```

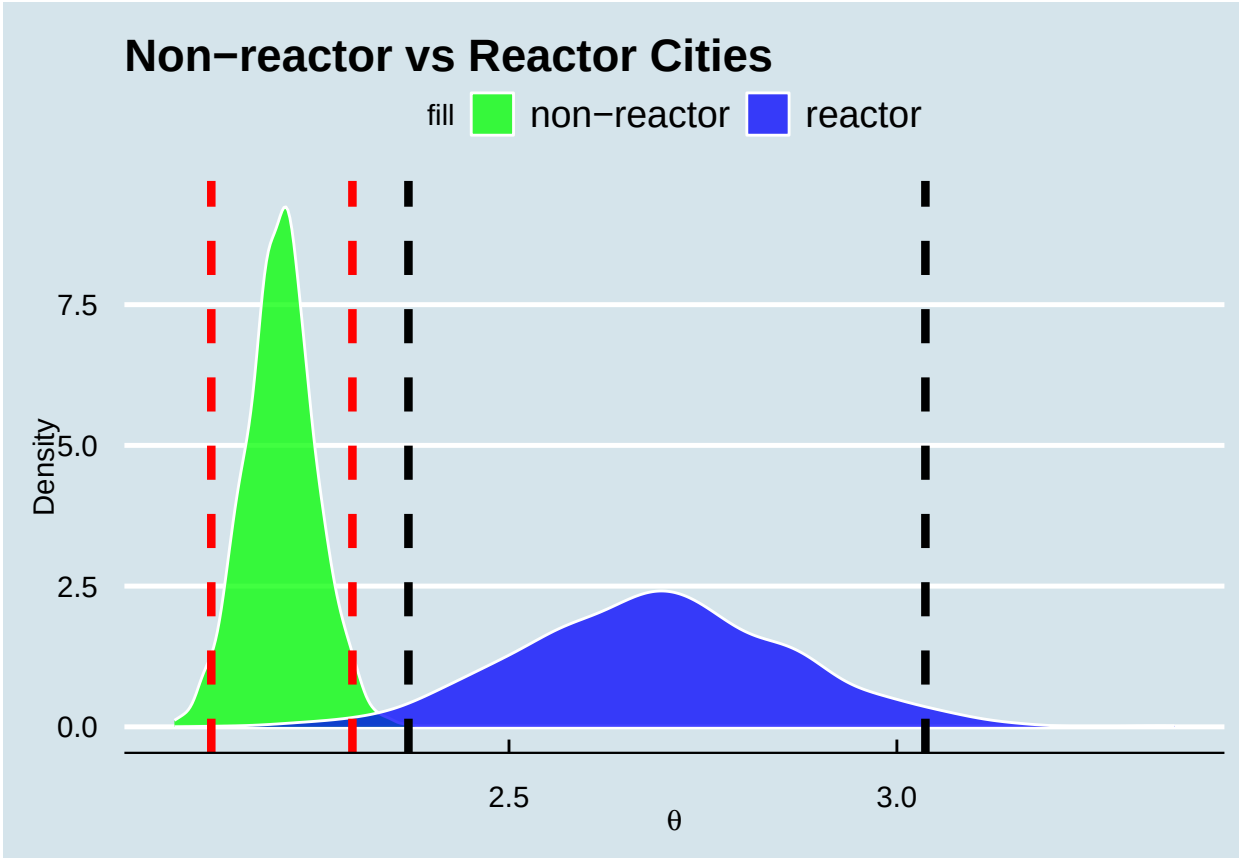
```
# setting seed
set.seed(27)

# required posterior probability estimate
```

```r
(
`Pr(theta2 > theta1 | data)` <- mean(
  x = (
    rgamma(n = 1000,
           shape = (a2+256),
           rate = (b2+95)) > rgamma(n = 1000,
                                    shape = (a1+2285),
                                    rate = (b2+1037))
  )
)
)
```

```
## [1] 0.998
```

```r
##########################################################

# setting seed
set.seed(27)

# data
dta <- tibble(
  # non-reactor cities
  nonreactor = rgamma(
    n = 1000,
    shape = (a1+2285),
    rate = (b1+1037)
  ),

  # reactor cities
  reactor = rgamma(
    n = 1000,
    shape = (a2+256),
    rate = (b2+95)
  )
)

# graphics
dta %>%
  ggplot() +

  # M.C. approximation of 1st posterior
  geom_density(aes(x = nonreactor,
                   fill = "non-reactor"),
               color = "white",
               alpha = 0.75) +

  # M.C. approximation of 2nd posterior
  geom_density(aes(x = reactor,
                   fill = "reactor"),
               color = "white",
               alpha = 0.75) +

  # confidence interval for non-reactor cities
```

```r
geom_vline(
  xintercept = dta$nonreactor %>%
    quantile(probs = c(0.025, 0.975)),
  color = "red",
  linetype = "dashed",
  linewidth = 1.5
) +

# confidence interval for reactor cities
geom_vline(
  xintercept = dta$reactor %>%
    quantile(probs = c(0.025, 0.975)),
  color = "black",
  linetype = "dashed",
  linewidth = 1.5
) +

scale_fill_manual(
  values = c(
    "non-reactor" = "green",
    "reactor" = "blue"
  )
) +
labs(title = "Non-reactor vs Reactor Cities") +
xlab(label = expression(theta)) +
ylab(label = "Density") +
theme_economist()
```



Conclusion

In all cases, it appears that non-reactor cities exhibit a lot more dispersion than the reactor cities.

Ex-4.5(d)

No, it's not reasonable. It's unlikely that areas with high fatality rates would be densely populated. Let's think of a contour plot. In an ideal world, for any decidedly high fatality rate, the joint likelihood would decrease as population size increases because people would naturally select against living in such locations (due to health concerns and whatnot).

But how would the analysis have to change??

Well, Kaoru Babasaki mentions hierarchical model. For instance, modelling θ_i as a function of population size, X_i. I'll look further into hierarchical models later.

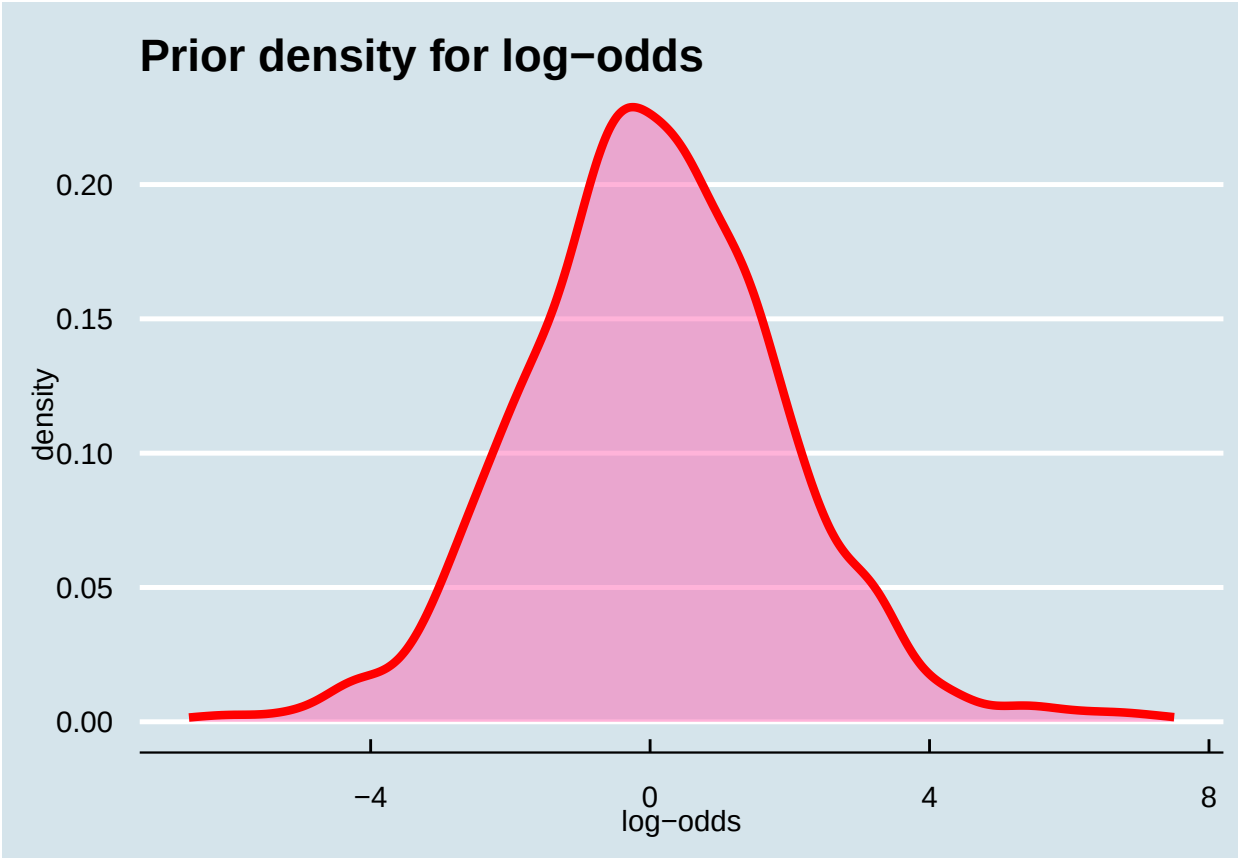
Ex-4.5(e)

We could establish *a priori* dependence between θ_1 and $theta_2$ by implementing weighted mixtures of the pure priors, $Gamma(a_1,\ b_1)$ and $Gamma(a_2,\ b_2)$.

For example: $\theta_i \sim \omega_i * Gamma(a_1,\ b_1) + (1 - \omega_i) * Gamma(a_2,\ b_2)$.

Ex-4.6

```r
# defining logit function
logit <- function(x){

  return(log(x/(1-x)))

}

set.seed(27)

# Monte-Carlo sampling uniform prior and transforming data
`gamma` <- logit(x = runif(n = 1000))

# plotting prior on log-odds scale
tibble(
  logodds = `gamma`
) %>%
  ggplot(aes(x = logodds)) +
  geom_density(
    linewidth = 1.5,
    color = "red",
    fill = "hotpink",
    alpha = 0.5
  ) +
  xlab(label = "log-odds") +
  labs(
    title = "Prior density for log-odds"
  ) +
  theme_economist()
```



Conclusion

The prior for log-odds appears to be approximately normally distributed. And so, while the prior is uninformative about θ, it **is informative** about $\gamma = log(\frac{\theta}{1-\theta})$.

Ex-4.7

Ex-4.7(a)

The appropriate **multinomial distribution** will be used to randomly pick one of the **relevant** pure posterior predictive distributions.

Variance will be generated from the given **marginal posterior distribution**. Then the mean will be generated from the given **full conditional distribution** based on the previously simulated variance.

Using both parameters, y will be simulated from the relevant **pure** posterior predictive distribution.

Asymptotically, the ys will appear to have been sampled from a tri-normal mixture distribution.

```
set.seed(27)

mat <- rmultinom(
  n = 5000, # number of draws within the simulation
  size = 1, # number of trials within each draw
  prob = c(0.31,0.46,0.23) # selection probabilities
```

```r
)

# object for storing Monte-Carlo samples
y.mc <- NULL

# generating parameters
VAR <- 1/rgamma(n = 5000, shape = 10, rate = 2.5)

MU <- sapply(X = VAR,
             FUN = function(x){
               rnorm(n = 1, mean = 4.1, sd = sqrt(x))
             })

for(i in 1:5000){

  if(mat[1, i] == 1){

    y.mc[i] <- rnorm(n = 1, mean = MU, sd = sqrt(VAR))

  }
  else if(mat[2, i] == 1){

    y.mc[i] <- rnorm(n = 1, mean = 2*MU, sd = 2*sqrt(VAR))

  }
  else{

    y.mc[i] <- rnorm(n = 1, mean = 3*MU, sd = 3*sqrt(VAR))

  }

}
```

Ex-4.7(b)

```r
# 75% quantile-based confidence interval
(
ci <- c(
  lower = max(y.mc[y.mc <= quantile(x = y.mc, probs = 0.125)]),
  upper = min(y.mc[y.mc >= quantile(x = y.mc, probs = 0.875)])
)
)
```

```
##     lower     upper
##  4.794561 14.563034
```

Ex-4.8

Ex-4.8(a)

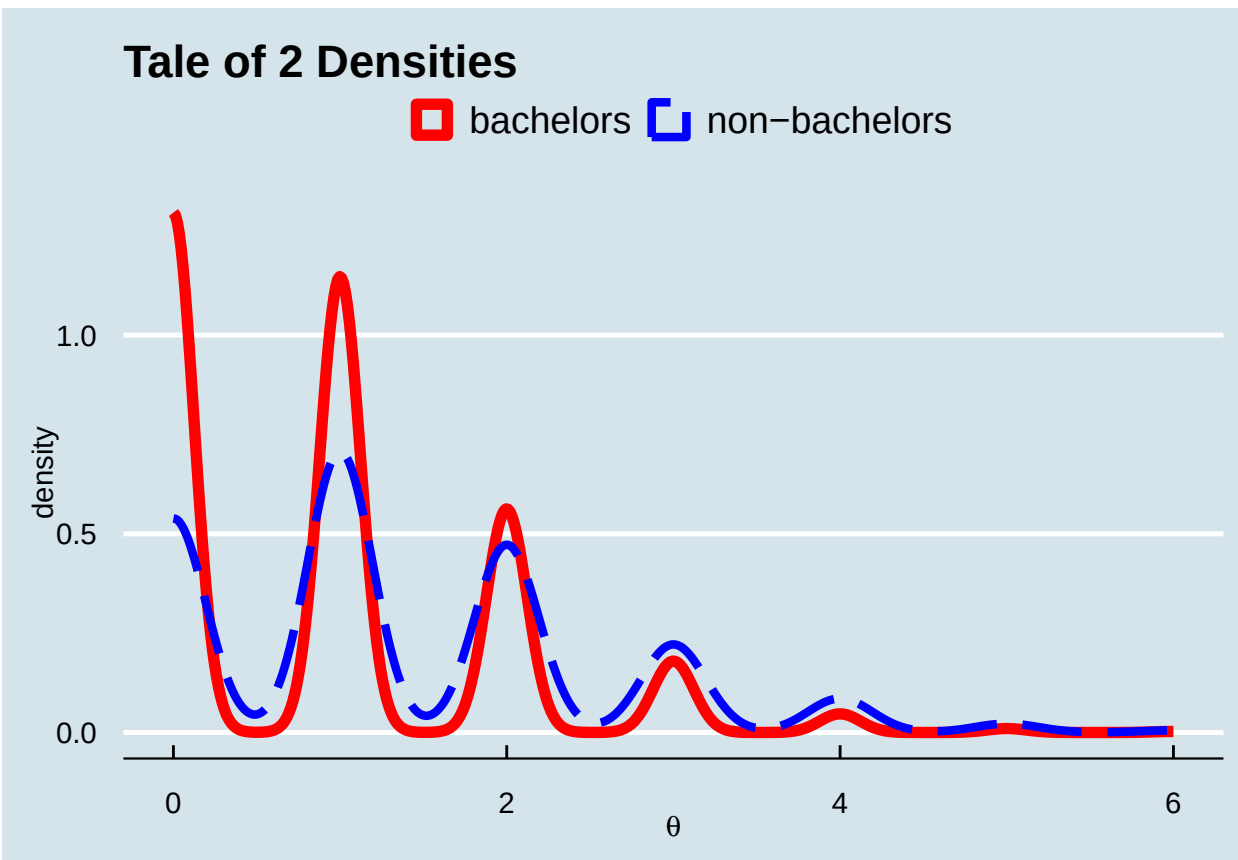
The posterior predictive distribution, in this case, is obviously a negative binomial distribution. I will spare myself the derivation because I already did it when I was studying earlier chapters. The general form of the posterior distribution is $N.B.(r = a + (\sum_{i=1}^{n} y_i),\ p = \frac{(b+n)}{(b+n+1)})$.

```
menchild30bach <- c(1, 0, 0, 1, 2, 2, 1, 5, 2, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 2, 1, 3, 2, 0,

menchild30nobach <- c(2, 2, 1, 1, 2, 2, 1, 2, 1, 0, 2, 1, 1, 2, 0, 2, 2, 0, 2, 1, 0, 0, 3, 6, 1, 6, 4, 
```

```
# hyper-parameters
a <- 2; b <- 1

# necessary posterior stats
sumA <- menchild30bach %>% sum()
sumB <- menchild30nobach %>% sum()

nA <- menchild30bach %>% length()
nB <- menchild30nobach %>% length()

N <- 5000

set.seed(27)

# predictive samples from A
mcA <- rnbinom(n = N,
               size = (a+sumA),
               prob = ((b+nA)/(b+nA+1)))

# predictive samples from B
mcB <- rnbinom(n = N,
               size = (a+sumB),
               prob = ((b+nB)/(b+nB+1)))
```

```
# graphics
tibble(
  sampleA = mcA,
  sampleB = mcB,
  order = 1:N
) %>%
  ggplot() +
  geom_density(aes(x = sampleA,
                   color = "bachelors",
                   linetype = "bachelors"),
               linewidth = 2) +
  geom_density(aes(sampleB,
                   color = "non-bachelors",
                   linetype = "non-bachelors"),
               linewidth = 1.5) +
  scale_color_manual(
    values = c(
```

```
    "bachelors" = "red",
    "non-bachelors" = "blue"
  )
) +
scale_linetype_manual(
  values = c(
    "bachelors" = "solid",
    "non-bachelors" = "longdash"
  )
) +
labs(
  color = "",
  linetype = "",
  title = "Tale of 2 Densities"
) +
xlab(label = expression(theta)) +
theme_economist()
```



Conclusion

Men without bachelors are predicted as less likely to have fewer than 3 children and they are to be more likely than others at having 3 to 5 children. Beyond that, the densities appear to converge.

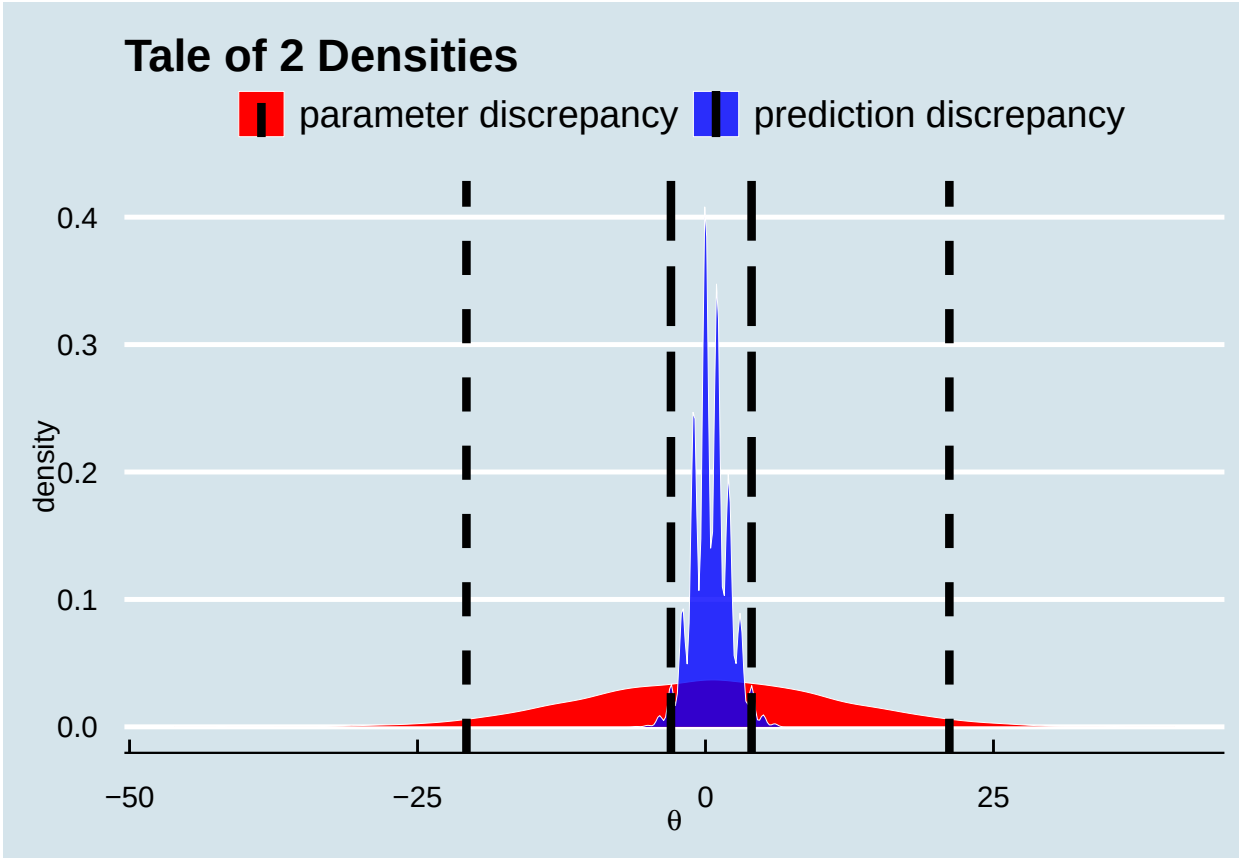
4.8(b)

I'm gonna spare myself from deriving the posterior distributions for θ_A and θ_B. These can easily be performed using knowledge from the earlier chapters.

```
# posterior parameters
shape.post.A <- (a+sumA); shape.post.B <- (a+sumB)
rate.post.A <- (b+nA)/(b+nA+1); rate.post.B <- (b+nB)/(b+nB+1)

set.seed(27)

# MC sampling
mc.theta.A <- rgamma(n = N,
                     shape = shape.post.A,
                     rate = rate.post.A)

mc.theta.B <- rgamma(n = N,
                     shape = shape.post.A,
                     rate = rate.post.A)

# posterior predictive sampling
mc.y.A <- mcA
mc.y.B <- mcB

# differences
diff.theta <- mc.theta.B - mc.theta.A
diff.y <- mc.y.B - mc.y.A

# 95% confidence intervals
ci.theta <- quantile(x = diff.theta,
                     probs = c(0.025, 0.975))

ci.y <- quantile(x = diff.y,
                 probs = c(0.025, 0.975))
```

```
# graphics
tibble(
  diff.theta = diff.theta,
  diff.y = diff.y
) %>%
  ggplot() +
  geom_density(aes(x = diff.theta,
                   fill = "parameter discrepancy",
                   color = "parameter discrepancy"),
               linewidth = 0.1) +
  geom_density(aes(x = diff.y,
                   fill = "prediction discrepancy",
                   color = "prediction discrepancy"),
               linewidth = 0.1,
               alpha = 0.8) +
  geom_vline(data = tibble(ci.theta = ci.theta),
             aes(xintercept = ci.theta,
                 linetype = "parameter discrepancy"),
             linewidth = 1.5) +
```

```r
geom_vline(data = tibble(ci.y = ci.y),
           aes(xintercept = ci.y,
               linetype = "prediction discrepancy"),
           linewidth = 1.5) +
xlim(c(min(diff.theta), max(diff.theta))) +
scale_fill_manual(
  values = c(
    "parameter discrepancy" = "red",
    "prediction discrepancy" = "blue"
  )
) +
scale_color_manual(
  values = c(
    "parameter discrepancy" = "white",
    "prediction discrepancy" = "white"
  )
) +
scale_linetype_manual(
  values = c(
    "parameter discrepancy" = "dashed",
    "prediction discrepancy" = "longdash"
  )
) +
labs(
  color = "",
  fill = "",
  linetype = "",
  title = "Tale of 2 Densities"
) +
xlab(label = expression(theta)) +
theme_economist()
```



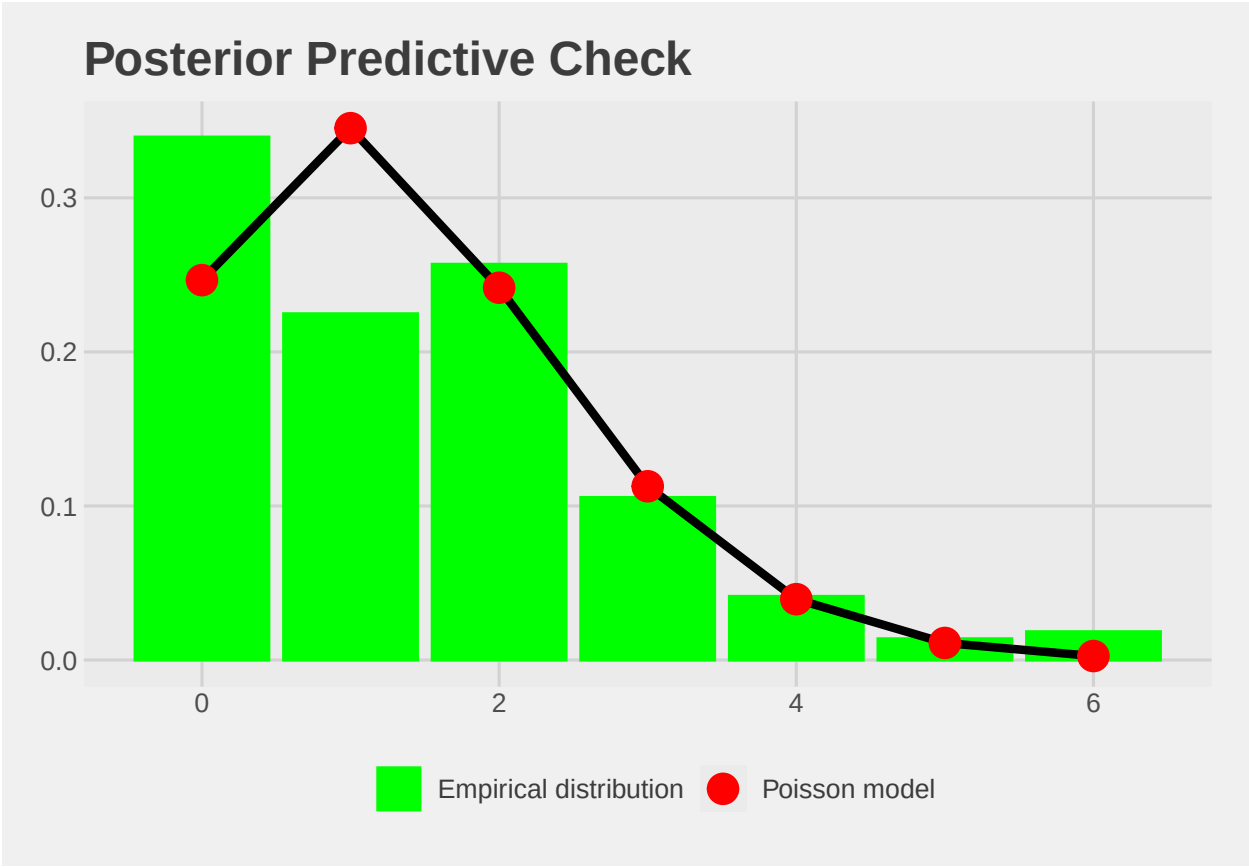
Conclusion

The prediction discrepancies have a smaller smaller spread than the parameter discrepancies. This goes to show that **just because there is a difference does not mean that the difference itself is large**.

Ex-4.8(c)

```
tibble(
  groupB = menchild30nobach
) %>%
  ggplot() +
  geom_bar(aes(x = groupB, y = after_stat(count/sum(count)),
               color = "Empirical distribution",
               fill = "Empirical distribution")) +
  geom_line(data = tibble(X = 0:6,
                          Y = dpois(x = X, lambda = 1.4)),
            aes(x = X, y = Y),
            linewidth = 1.5) +
  geom_point(data = tibble(X = 0:6,
                           Y = dpois(x = X, lambda = 1.4)),
             aes(x = X, y = Y,
                 color = "Poisson model",
                 fill = "Poisson model"),
```

```r
            size = 5) +
scale_color_manual(
  values = c(
    "Poisson model" = "red",
    "Empirical distribution" = "green"
  )
) +
scale_fill_manual(
  values = c(
    "Poisson model" = "red",
    "Empirical distribution" = "green"
  )
) +
labs(
  color = "",
  fill = "",
  title = "Posterior Predictive Check"
) +
xlab(label = "Number of children") +
ylab(label = "density") +
theme_fivethirtyeight()
```



Conclusion

The Poisson model is **not a good fit**. The empirical distribution fluctuates between 0 and 2 children whereas the Poisson model peaks at 1 child. Afterwards, both decline steadily in terms of density.